



SALPHOENIX®2A 系列 FPGA
PCI Express Native 接口转 AXI Stream
(RQ, RC, CQ, CC) 接口参考设计

上海安路信息科技股份有限公司

APUG089 (v1.0) 2024 年 05 月



目 录

1 概要	3
1.1 文件和目录结构	3
2 参考设计概述	5
2.1 验证环境	5
2.2 功能描述	5
2.2.1 系统框图	5
2.2.2 验证范围	6
2.2.3 使用资源	7
3 功能概述	8
3.1 系统结构	8
3.2 AXIS 接口描述	9
3.2.1 CQ 接口	9
3.2.2 RC 接口	12
3.2.3 CC 接口	15
3.2.4 RQ 接口	17
3.3 顶层信号	21
4 仿真	22
4.1 运行仿真	22
4.2 仿真结构	23
5 编译与调试	25
5.1 配置说明	25
5.2 RC 工程说明	25
5.3 RC 连接 EP 测试过程	27



5.3.1 TLP 测试过程.....	28
5.4 PC 连接 EP 测试过程.....	29
5.4.1 TLP 测试过程.....	30
6 工程移植注意事项.....	32
7 参考文献.....	37
版本信息.....	38
免责声明.....	38



1 概要

本文档提供了 SALPHOENIX®2A (以下简称 PH2A) 系列 FPGA 的 PCIe 接口转换工程说明, 以便于用户快速掌握 PH2A 系列器件 PCIe Native 接口转 AXI Stream 接口的使用。该工程将 PCIe 原生的数据接口 (tl_rx_*, tl_tx_*) 转换为 CQ (Completer request)、CC (Completer completion)、RQ (Requester request)、RC (Requester completion) 接口, 便于用户将其他厂家器件移植到安路 FPGA 器件。

在本参考设计中, 提供两种验证工程, 一是在 RC 对接 EP 方式下, 配合串口驱动验证 CQ、CC 接口功能, PH1A 系列 demo 板配置为 RC 模式, PH2A 系列 demo 板配置为 EP 模式; 二是配合 Linux 驱动验证 RQ、RC 接口功能, PH2A 系列 demo 板配置为 EP 模式, PC 机作为 RC。对于 Posted 报文, 将有效数据载荷存到 BRAM 中; 对于 Non-Posted 报文, 用户逻辑发送 CPLD 包, RC 收到 CPLD 包并进行数据校验, 如果数据校验成功, 证明该系统工作正常。

如果用户只需要接口转换的功能, 而不需要其他的验证逻辑, Design/case_ep/src/native2axis_ip/src 目录下为接口转换的代码。

1.1 文件和目录结构

参考设计包含了所有需要的文件, 包括 TD 工程等, 参考设计文件结构如下:

表 1-1 参考设计结构表

目录		说明		
Doc		参考设计相关的文档		
Design		RC 和 EP 的设计文件		
	bit		bit 文件	
	case_ep		PCIe EP 工程	
		impl		EP 的 TD 工程
		src		EP 工程源文件
		tb		EP 工程 testbench
	case_rc		PCIe RC 工程	
		impl		RC 的 TD 工程
		src		RC 工程源文件
	Linux driver		Linux 系统测试驱动	



	case_rq_rc		PC 连接 EP 测试驱动
		anlogic_pcie_drv	PCIe 测试驱动
		anlogic_pcie_app	PCIe 测试程序
	case_cq_cc		RC 对接 EP 测试驱动
		uart2pcie	PCIe 测试驱动和程序



2 参考设计概述

使用本文档对应的参考设计，能够在安路提供的 PH2A 系列 Demo 板 AP201 上完成 PCIe Native 接口转 AXI Stream 的简单验证。

参考设计提供 PH2A EP gen4x8 的工程。可直接对接 PC，另外提供了 PH2A EP gen3x4 的 bit，也可以用于连接本参考设计提供的 PH1A gen3x4 的 RC 工程进行测试。

同时，参考设计提供了完整的仿真设计。

2.1 验证环境

参考设计使用软硬件版本如下：

表 2-1 参考设计软件环境

Name	Version
FPGA	PH2A106FLG676
TD Software	软件 6.0.2 108839 以及后续版本
OS	Ubuntun 22.04.1 LTS

2.2 功能描述

2.2.1 系统框图

参考设计系统框图如下：

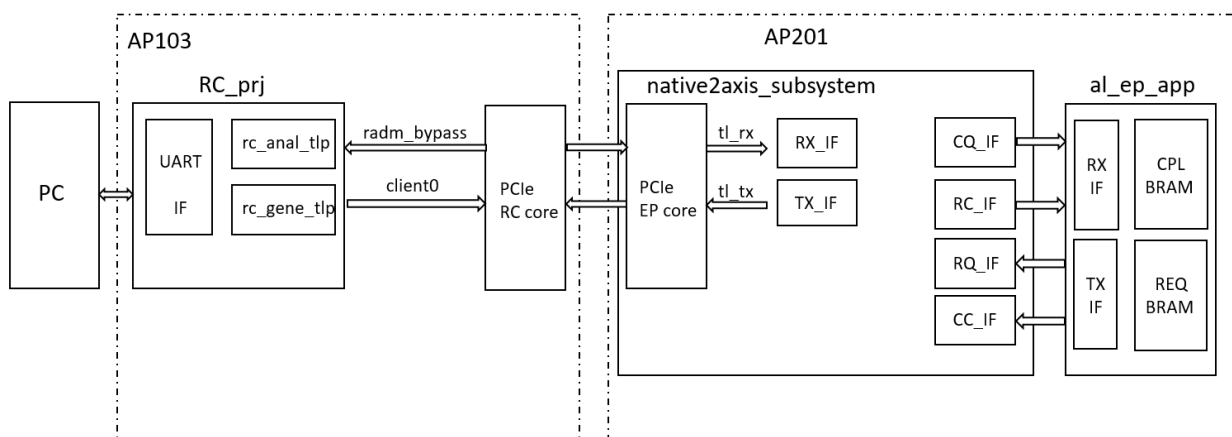


图 2-1 参考设计 RC 对接 EP 系统框图



在 RC 对接 EP 工程中，上位机通过测试脚本和串口控制 PCIe RC 工程发起 CFGWR0/CFGRD0/MEMWR/MEMRD 这些 request，组包之后通过 PCIe RC core 发出。对端 PCIe ep core 收到 request 后，CFGWR0/CFGRD0 直接被 pcie_ep_core 接收，MEMWR/MEMRD 通过 tl_rx_* 和 tl_tx_* 接口经过 native2axis_subsys 模块处理后，转换成 CQ、CC、RQ、RC 总线，送到 al_ep_app 模块。al_ep_app 模块通过 CQ CC 总线接收 MEMWR/MEMRD 并返回完成包，通过 RQ、RC 总线发送 MEMWR/MEMRD 并接收完成包。

在 PC 对接 EP 方式下，上位机通过 linux 测试驱动，向 EP 端发起 MEMWR/MEMRD 请求，EP 端接收到请求包后转换成 CQ、CC、RQ、RC 总线，送到 al_ep_app 模块。al_ep_app 模块通过 CQ CC 总线接收 MEMWR/MEMRD 并返回完成包，通过 RQ、RC 总线发送 MEMWR/MEMRD 并接收完成包。

2.2.2 验证范围

目前 PCIe Native 接口转 AXI Stream 仅支持实现在 EP 端，下面表 2-2 列出 EP 支持收发的报文：

表 2-2 EP 支持收发的报文

接收 TLP	发送 TLP
Configuration Read Type 0 Request (CFGRD0)	Memory Write Request (MEMWR)
Configuration Write Type 0 Request (CFGWR0)	Memory Read Request (MEMRD)
Memory Write Request (MEMWR)	Completion without Data (CPL)
Memory Read Request (MEMRD)	Completion with Data (CPLD)
Completion without Data (CPL)	
Completion with Data (CPLD)	

在 RC 对接 EP 测试过程中，RC 通过 UART 接收命令，并转成相应的 PCIe TLP 报文，与 EP 端进行交互，主要包含以下测试报文：

表 2-3 RC 工程支持发出的报文

Type	Description
cfg0wr	使能 EP 的 Memory 和 IO 空间
	设置 BAR0 的基地址
cfg0rd	读取 EP 的 Device ID
	读取 BAR0 的基地址
Memwr	向 BAR 地址写任意长度数据



Memrd	向 BAR 地址读任意长度数据
-------	-----------------

2.2.3 使用资源

工程使用资源情况如下：

表 2-4 EP 资源使用

名称	用量	百分比
Lut	6084	2.47%
Slice	10813	2.19%
Eram	157	13.08%



3 功能概述

3.1 系统结构

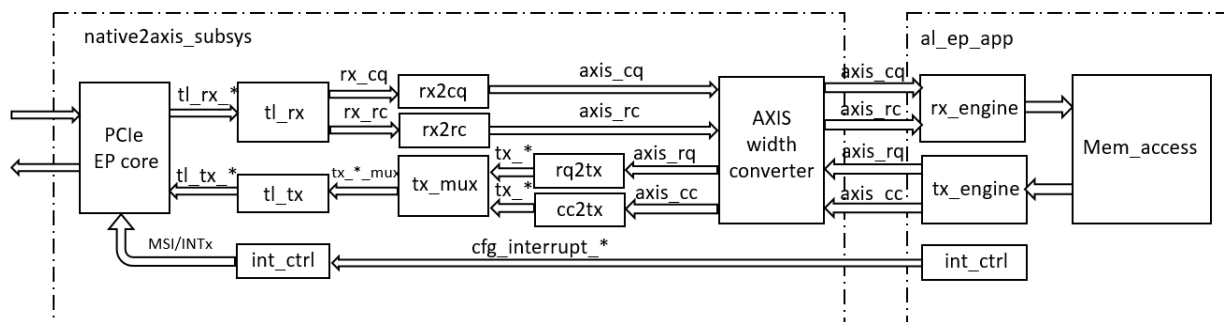


图 3-1 EP 端系统框图

其中，al_ep_app 模块中实现了简单的 block dma 控制器，RC 能够通过 CQ 接口配置其中的 DMA 寄存器，使得 al_ep_app 模块主动通过 RQ、RC 接口发起 h2c 和 c2h DMA 传输。DMA 寄存器定义如表 3-2 所示。

表 3-2 DMA 寄存器列表

寄存器名称	寄存器偏移地址	Bit 位置	描述
h2c_start	0x4	bit[0]	DMA h2c 开始信号，高有效
c2h_start	0x4	bit[1]	DMA c2h 开始信号，高有效
h2c 基地址	0x1000	bit[31:0]	DMA h2c 基地址
h2c 传输字节长度	0x1004	bit[31:0]	DMA h2c 传输字节长度
c2h 基地址	0x1800	bit[31:0]	DMA c2h 基地址
c2h 传输字节长度	0x1804	bit[31:0]	DMA c2h 传输字节长度

EP 里，有 1 个 BAR，具体设置如下表，也可以配置其他合理值。

表 3-3 EP BAR 配置表

BAR NUM	BAR0
Memory/I/O	Memory



Non-prefetchable/Prefetchable	prefetchable
32bit/64bit	64bit
Space size	2 ⁰ MBytes

al_ep_app 中 BAR 空间划分为 DMA 空间和 USER 空间，DMA 空间偏移地址为 0x0~0x2000，其余空间为 USER 空间。USER 空间 BAR 的实现由 FPGA 内部 Fabric 中 BRAM 完成的。

3.2 AXIS 接口描述

EP 端存在四组接口转换模块，分别是 rx2cq 模块，rx2rc 模块，cc2tx 模块，rq2tx 模块，实现了将 PCIe core tl_rx_* 总线和 tl_tx_* 总线到 AXIS CQ, RC, CC, RQ 总线的接口转换，下面将分别介绍这四组模块和接口。

3.2.1 CQ 接口

rx2cq 模块接收 PCIe 链路上发来的读写请求。PCIe IP 核接收到读写请求后，会通过 tl_rx 模块将接收到的 TLP 包转换成 CQ 接口的 AXIS 包格式送到 rx2cq 模块，该模块会先将数据存储在内部的 FIFO 中，当存储到完整一帧报文后，就会开始数据接口的转换，将 tl_rx_* 总线数据转换到 CQ 数据接口，并发送到用户侧。其中，AXIS 数据位宽支持 512bit/256bit/128bit/64bit。

表 3-4 CQ 接口端口定义

端口	I/O	位宽	说明
m_axis_cq_tdata	输出	512/256/128/64	来自 PCIe 链路的数据，输出到用户逻辑
m_axis_cq_tuser	输出	183/88	与正在传输的数据相关的边带信号，CQ 数据位宽为 512bit 时，该组信号位宽为 183bit；CQ 数据位宽为 256bit/128bit/64bit 时，该组信号位宽为 88bit，信号定义见表 3-5 和表 3-6
m_axis_cq_tlast	输出	1	CQ 数据最后一拍标志
m_axis_cq_tkeep	输出	16/8/4/2	CQ 数据每个 DWord 使能标志
m_axis_cq_tvalid	输出	1	CQ 数据有效信号
m_axis_cq_tready	输入	1	CQ 端口接收数据准备信号

其中，m_axis_cq_tuser 端口包含了与正在传输的数据相关的边带信号，信号定义如下表所示。

表 3-5 m_axis_cq_tuser 信号定义（183bit）



位索引	信号名	位宽	描述
3:0	first_be	4	第一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
7:4		4	Not used（在用户逻辑中可悬空）
11:8	last_be	4	最后一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
15:12		4	Not used（在用户逻辑中可悬空）
79:16	byte_en	64	数据载荷的字节有效信号，在 AXIS 包头描述符部分无效
80	sop	1	AXIS 包开始标志信号
85: 81		5	Not used（在用户逻辑中可悬空）
86	eop	1	AXIS 包结束标志信号
95: 87		9	Not used（在用户逻辑中可悬空）
96	discontinue	1	AXIS 包中出现错误标志信号
97	tph_present	1	PCIe 链路 TLP 包中的 TH 字段
98		1	Not used（在用户逻辑中可悬空）
100:99	tph_type	2	PCIe 链路 TLP 包中的 PH 字段
102:101		2	Not used（在用户逻辑中可悬空）
110:103	tph_st_tag	8	PCIe 链路 TLP 包中的 Steering Tag 字段
118:111		8	Not used（在用户逻辑中可悬空）
182:119	parity	64	m_axis_cq_tdata 数据的奇校验码

表 3-6 m_axis_cq_tuser 信号定义（88bit）

位索引	信号名	位宽	描述
3:0	first_be	4	第一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
7:4	last_be	4	最后一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效



39:8	byte_en	64	数据载荷的字节有效信号，在 AXIS 包头描述符部分无效
40	sop	1	AXIS 包开始标志信号
41	discontinue	1	AXIS 包中出现错误标志信号
42	tph_present	1	PCIe TLP 包中的 TH 字段
44:43	tph_type	2	PCIe TLP 包中的 PH 字段
52:45	tph_st_tag	8	PCIe TLP 包中的 Steering Tag 字段
84:53	parity	64	m_axis_cq_tdata 数据的奇校验码
87:85		3	Not used（在用户逻辑中可悬空）

rx2cq 模块将 PCIe TLP 包转换为 CQ 接口的 AXIS 包格式,AXIS 包由 128bit 描述符+数据载荷组成,描述符格式和 CQ 接口时序如图所示。

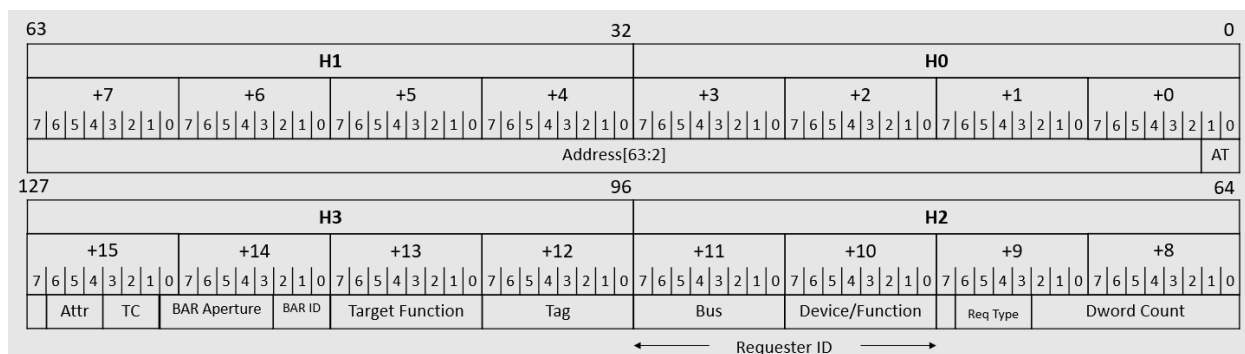


图 3-7 CQ AXIS 包头格式

表 3-8 CQ AXIS 包头信号定义

位索引	信号名	位宽	描述
1:0	address type	2	PCIe TLP 中 AT 字段
63:2	address	62	PCIe TLP 中 address 字段
74:64	dword count	11	PCIe TLP 中 dword count 字段
78:75	req type	4	标志请求包类型 0000: MEM Read 请求 0001: MEM Write 请求
79		1	Not Used（在用户逻辑中可悬空）



95:80	requester ID	16	PCIe TLP 中 requester ID 字段
103:96	tag	8	PCIe TLP 中 tag 字段
111:104	target function	8	目标 function
120:115		6	Not Used（在用户逻辑中可悬空）
123:121	TC	3	PCIe TLP 中 TC 字段
126:124	attr	3	PCIe TLP 中 attr 字段
127		1	Not Used（在用户逻辑中可悬空）

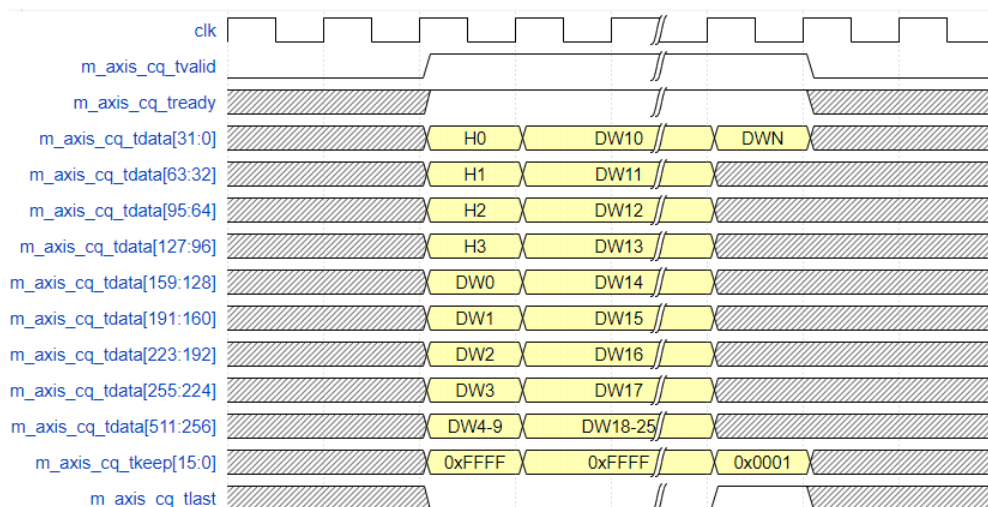


图 3-9 CQ 接口时序图 (512bit)

3.2.2 RC 接口

rx2rc 模块接收 PCIe 链路上发来的完成包，包括 CPL/CPLD 请求。PCIe IP 核接收到完成包之后，会通过 tl_rx 模块将接收到的 TLP 包转换成 RC 接口的 AXIS 包格式送到 rx2rc 模块，该模块会先将数据存储到内部的 FIFO 中，当存储到完整一帧报文后，就会开始数据接口的转换，将 tl_rx_*总线数据转换到 rc 数据接口，并发送到用户侧。

表 3-10 rc 接口端口定义

端口	I/O	位宽	说明
m_axis_rc_tdata	输出	512/256/128/64	来自 PCIe 链路的数据，输出到用户逻辑
m_axis_rc_tuser	输出	161/75	与正在传输的数据相关的边带信号，RC 数据位宽为 512bit 时，该组信号位宽为 161bit；RC 数据位宽为 256bit/128bit/64bit 时，该



			组信号位宽为 75bit，信号定义见表 3-11 和表 3-12
m_axis_rc_tlast	输出	1	RC 数据最后一拍标志
m_axis_rc_tkeep	输出	16/8/4/2	RC 数据每个 DWord 使能标志
m_axis_rc_tvalid	输出	1	RC 数据有效信号
m_axis_rc_tready	输入	4	RC 端口接收数据准备信号

其中，m_axis_rc_tuser 端口包含了与正在传输的数据相关的边带信号，信号定义如下表所示。

表 3-11 m_axis_rc_tuser 信号定义（161bit）

位索引	信号名	位宽	描述
63:0	byte_en	64	数据载荷的字节有效信号，在 AXIS 包头描述符部分无效
64	sop	1	AXIS 包开始标志信号
75:65		11	Not used（在用户逻辑中可悬空）
76	eop	1	AXIS 包结束标志信号
95:77		7	Not used（在用户逻辑中可悬空）
96	discontinue	1	AXIS 包中出现错误标志信号
160:97	parity	64	m_axis_rc_tdata 数据的奇校验码

表 3-12 m_axis_rc_tuser 信号定义（75bit）

位索引	信号名	位宽	描述
31:0	byte_en	32	数据载荷的字节有效信号，在 AXIS 包头描述符部分无效
32	sop	1	AXIS 包开始标志信号
33		1	Not used（在用户逻辑中可悬空）
34	eop	1	AXIS 包结束标志信号
41:35		7	Not used（在用户逻辑中可悬空）
42	discontinue	1	AXIS 包中出现错误标志信号
74:43	parity	32	m_axis_rc_tdata 数据的奇校验码



rx2rc 模块将 PCIe TLP 包转换为 RC 接口的 AXIS 包格式，AXIS 包由 96bit 包头描述符+数据载荷组成，包头格式和 RC 接口时序如图所示。

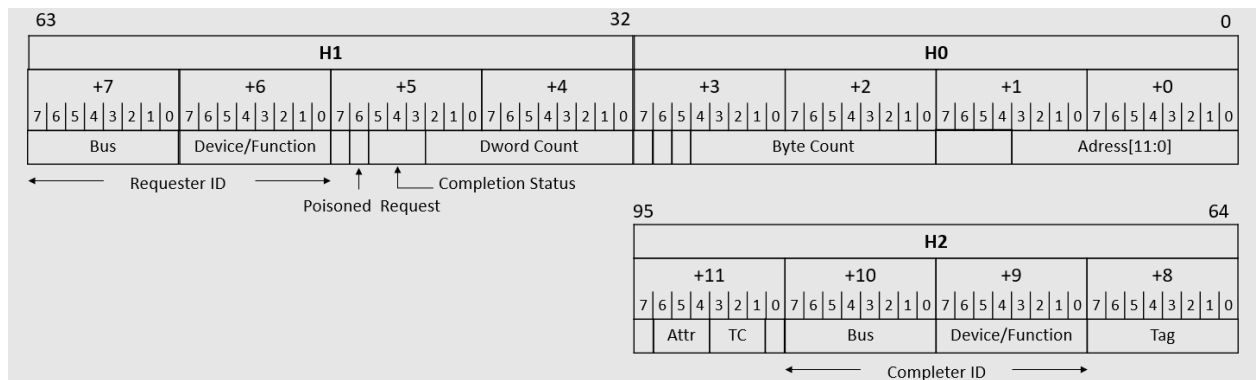


图 3-13 RC AXIS 包头格式

表 3-14 RC AXIS 包头信号定义

位索引	信号名	位宽	描述
11:0	lower address	12	PCIe TLP 中 lower address 字段
15:12		3	Not Used（在用户逻辑中可悬空）
28:16	byte count	13	PCIe TLP 中 byte count 字段
31:29		3	Not Used（在用户逻辑中可悬空）
42:32	dword count	11	PCIe TLP 中 dword count 字段
45:43	completion status	4	PCIe TLP 中 completion status 字段
46	poisoned completion	1	AXIS 包中出现错误标志信号
63:48	requester ID	16	PCIe TLP 中 requester ID 字段
71:64	tag	8	PCIe TLP 中 tag 字段
87:72	completer ID	16	PCIe TLP 中 completer ID 字段
88		1	Not Used（在用户逻辑中可悬空）
91:89	TC	3	PCIe TLP 中 TC 字段
94:92	attr	3	PCIe TLP 中 attr 字段
95		1	Not Used（在用户逻辑中可悬空）

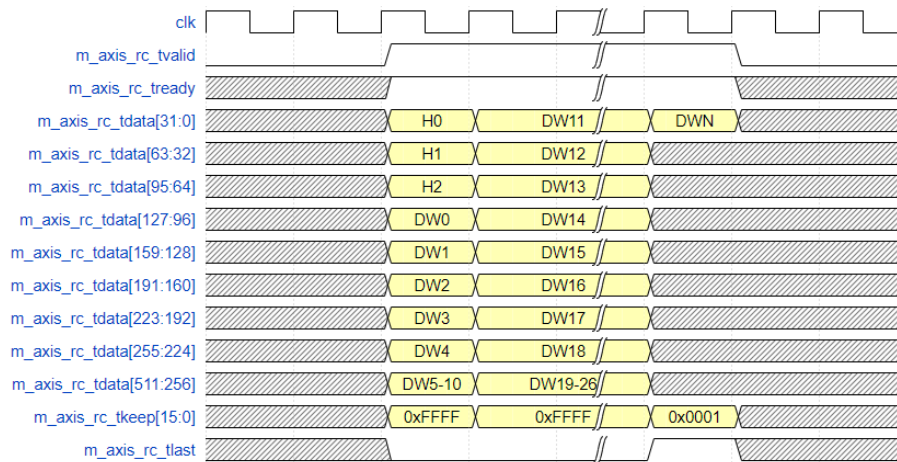


图 3-15 RC 接口时序图 (512bit)

3.2.3 CC 接口

cc2tx 模块接收由用户逻辑发来的完成包，包括 CPL/CPLD。接收到完成包之后，该模块会先将数据存储到内部的 FIFO 中，当存储到完整一帧报文后，就会开始数据接口的转换，通过 tl_tx 模块将接收到的 AXIS 包格式的完成包转换成 PCIe 的完成包 TLP 格式，然后通过 PCIe core 发送到对端 PCIe 设备。

表 3-16 CC 接口端口定义

端口	I/O	位宽	说明
s_axis_cc_tdata	输入	512/256/128/64	来自用户逻辑的 AXIS 数据，输出到 PCIe 链路
s_axis_cc_tuser	输入	81/33	与正在传输的数据相关的边带信号，CC 数据位宽为 512bit 时，该组信号位宽为 81bit；CQ 数据位宽为 256bit/128bit/64bit 时，该组信号位宽为 33bit，信号定义见表 3-17 和表 3-18
s_axis_cc_tlast	输入	1	CC 数据最后一拍标志
s_axis_cc_tkeep	输入	16/8/4/2	CC 数据每个 Dword 使能标志
s_axis_cc_tvalid	输入	1	CC 数据有效信号
s_axis_cc_tready	输出	1	CC 端口接收数据准备信号

表 3-17 s_axis_cc_tuser 信号定义 (81bit)



位索引	信号名	位宽	描述
0	sop	1	AXIS 包开始标志信号
5:1		5	Not used（在用户逻辑中可接 0）
6	eop	11	AXIS 包结束标志信号
15:7		9	Not used（在用户逻辑中可接 0）
16	discontinue	1	AXIS 包中出现错误标志信号
80:17	parity	64	s_axis_cc_tdata 数据的奇校验码

表 3-18 s_axis_cc_tuser 信号定义（33bit）

位索引	信号名	位宽	描述
0	discontinue	1	AXIS 包中出现错误标志信号
32:1	parity	32	s_axis_cc_tdata 数据的奇校验码

cc2tx 模块将 PCIe TLP 包转换为 CC 接口的 AXIS 包格式，AXIS 包由 96bit 描述符+数据载荷组成，描述符格式和 CC 接口时序如图所示。

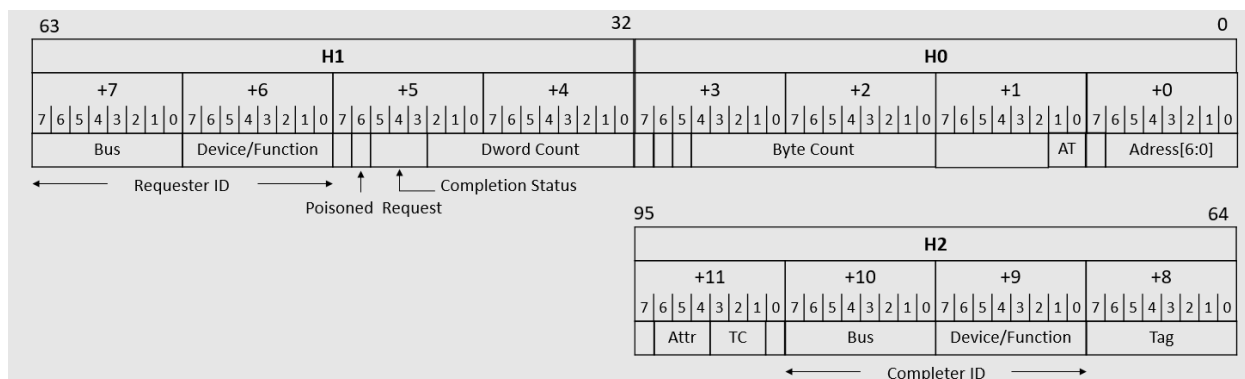


图 3-19 CC AXIS 包头格式

表 3-20 CC AXIS 包头信号定义

位索引	信号名	位宽	描述
6:0	lower address	7	PCIe TLP 中 lower address 字段
7		1	Not Used（在用户逻辑中可接 0）
9:8	address type	2	PCIe TLP 中 AT 字段



15:10		6	Not Used（在用户逻辑中可接 0）
28:16	byte count	13	PCIe TLP 中 byte count 字段
31:29		3	Not Used（在用户逻辑中可接 0）
42:32	dword count	11	PCIe TLP 中 dword count 字段
45:43	completion status	3	PCIe TLP 中 completion status 字段
46	poisoned completion	1	AXIS 包中出现错误标志信号
47		1	Not Used（在用户逻辑中可接 0）
63:48	requester ID	16	PCIe TLP 中 requester ID 字段
71:64	tag	3	PCIe TLP 中 tag 字段
87:72	completer ID	16	PCIe TLP 中 completer ID 字段
88		1	Not Used（在用户逻辑中可接 0）
91:89	TC	3	PCIe TLP 中 TC 字段
94:92	attr	3	PCIe TLP 中 attr 字段
95		1	Not Used（在用户逻辑中可接 0）

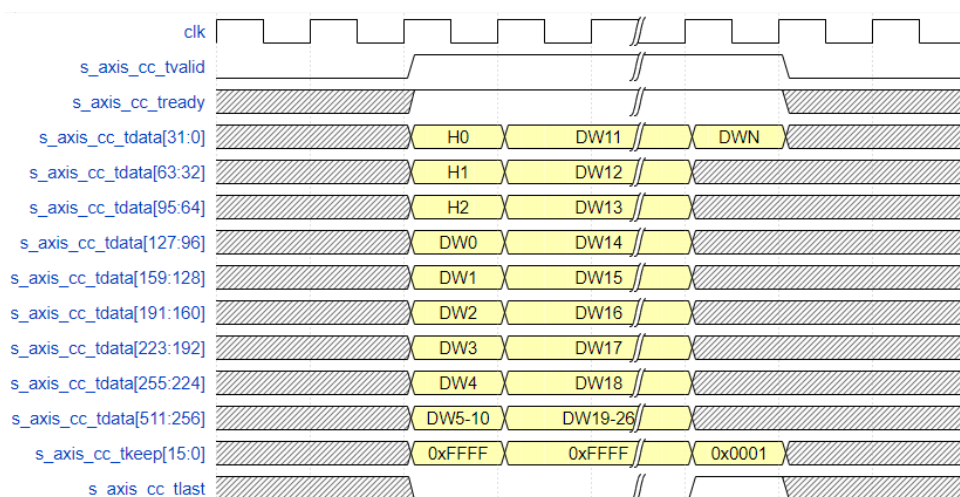


图 3-21 CC 接口时序图（512bit）

3.2.4 RQ 接口

rq2tx 模块接收由用户逻辑发来的读写请求包。接收到请求包之后，该模块会先将数据存储在内部



的 FIFO 中，当存储到完整一帧报文后，就会开始数据接口的转换，通过 `tl_tx` 模块将接收到的 AXIS 包格式的请求包转换成 PCIe 的 MEMW\MEMR TLP 格式，然后通过 PCIe core 发送到对端 PCIe 设备。

表 3-22 RQ 接口端口定义

端口	I/O	位宽	说明
<code>s_axis_rq_tdata</code>	输入	512/256/128/64	来自用户逻辑的 AXIS 数据，输出到 PCIe 链路
<code>s_axis_rq_tuser</code>	输入	137/62	与正在传输的数据相关的边带信号，RQ 数据位宽为 512bit 时，该组信号位宽为 137bit；RQ 数据位宽为 256bit/128bit/64bit 时，该组信号位宽为 62bit，信号定义见表 3-23 和表 3-24
<code>s_axis_rq_tlast</code>	输入	1	RQ 数据最后一拍标志
<code>s_axis_rq_tkeep</code>	输入	16/8/4/2	RQ 数据每个 DWord 使能标志
<code>s_axis_rq_tvalid</code>	输入	1	RQ 数据有效信号
<code>s_axis_rq_tready</code>	输出	1	RQ 端口接收数据准备信号
<code>pcie_rq_seq_num0</code>	输出	6	RQ 端口请求包序列号
<code>pcie_rq_seq_num_vld0</code>	输出	1	RQ 端口请求包序列号有效信号

表 3-23 `s_axis_rq_tuser` 信号定义（137bit）

位索引	信号名	位宽	描述
3:0	<code>first_be</code>	4	第一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
7:4		4	Not used（在用户逻辑中可接 0）
11:8	<code>last_be</code>	4	最后一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
19:12		8	Not used（在用户逻辑中可接 0）
20	<code>sop</code>	1	AXIS 包开始标志信号
25:21		5	Not used（在用户逻辑中可接 0）



26	eop	1	AXIS 包结束标志信号
35: 27		9	Not used (在用户逻辑中可接 0)
36	discontinue	1	AXIS 包中出现错误标志信号
37	tph_present	1	PCIe 链路 TLP 包中的 TH 字段
38		1	Not used (在用户逻辑中可接 0)
40:39	tph_type	2	PCIe 链路 TLP 包中的 PH 字段
44:41		2	Not used (在用户逻辑中可接 0)
52:45	tph_st_tag	8	PCIe 链路 TLP 包中的 Steering Tag 字段
66:61	seq_num0	8	用户逻辑提供的序列号, 可用于监控请求包在模块内部的处理过程, 当请求包将要被发出时, 该序列号会出现在 pcie_rq_seq_num0 接口
72:67		8	Not used (在用户逻辑中可接 0)
136:73	parity	64	s_axis_rq_tdata 数据的奇校验码

表 3-24 s_axis_rq_tuser 信号定义 (62bit)

位索引	信号名	位宽	描述
3:0	first_be	4	第一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
7:4	last_be	4	最后一个 Dword 数据载荷的字节有效信号。在 AXIS 包的第一拍有效
11	discontinue	1	AXIS 包中出现错误标志信号
12	tph_present	1	PCIe 链路 TLP 包中的 TH 字段
14:13	tph_type	2	PCIe 链路 TLP 包中的 PH 字段
15		1	Not used (在用户逻辑中可接 0)
23:16	tph_st_tag	8	PCIe 链路 TLP 包中的 Steering Tag 字段
27:24	seq_num[3:0]	4	用户逻辑提供的序列号, 可用于监控请求包在模块内部的处理过程, 当请求包将要被发出时, 该序列号会出现在 pcie_rq_seq_num0 接口
59:28	parity	32	s_axis_rq_tdata 数据的奇校验码



61:60	seq_num[5:4]	2	seq_num 的 bit[5:4]
-------	--------------	---	--------------------

rq2tx 模块将 PCIe TLP 包转换为 RQ 接口的 AXIS 包格式,AXIS 包由 128bit 描述符+数据载荷组成,描述符格式和 RQ 接口时序如图所示。

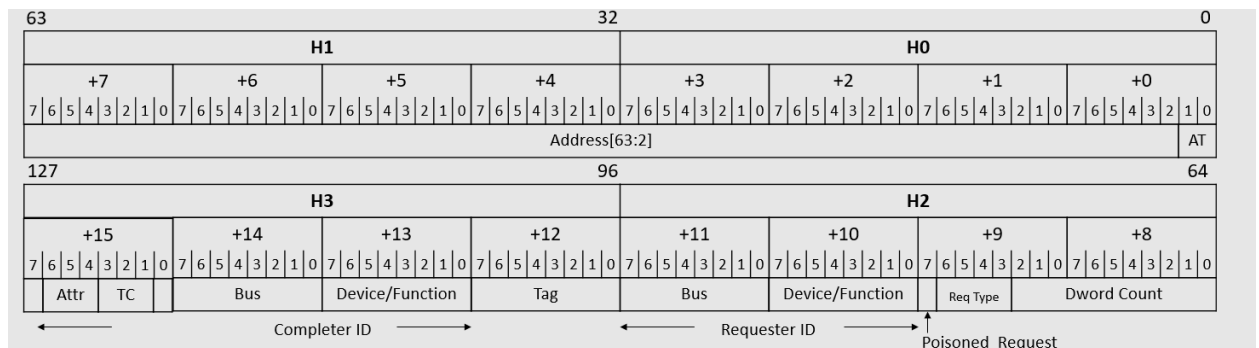


图 3-25 RQ AXIS 包头格式

表 3-26 RQ AXIS 包头信号定义

位索引	信号名	位宽	描述
1:0	address type	2	PCIe TLP 中 AT 字段
63:2	address	62	PCIe TLP 中 address 字段
74:64	dword count	11	PCIe TLP 中 dword count 字段
78:75	req type	4	标志请求包类型 0000: MEM Read 请求 0001: MEM Write 请求
79	poisoned req	1	AXIS 包中出现错误标志信号
95:80	requester ID	16	PCIe TLP 中 requester ID 字段
103:96	tag	8	PCIe TLP 中 tag 字段
119:104	completer ID	16	PCIe TLP 中 completer ID 字段
120		1	Not Used (在用户逻辑中可接 0)
123:121	TC	3	PCIe TLP 中 TC 字段
126:124	attr	3	PCIe TLP 中 attr 字段
127		1	Not Used (在用户逻辑中可接 0)

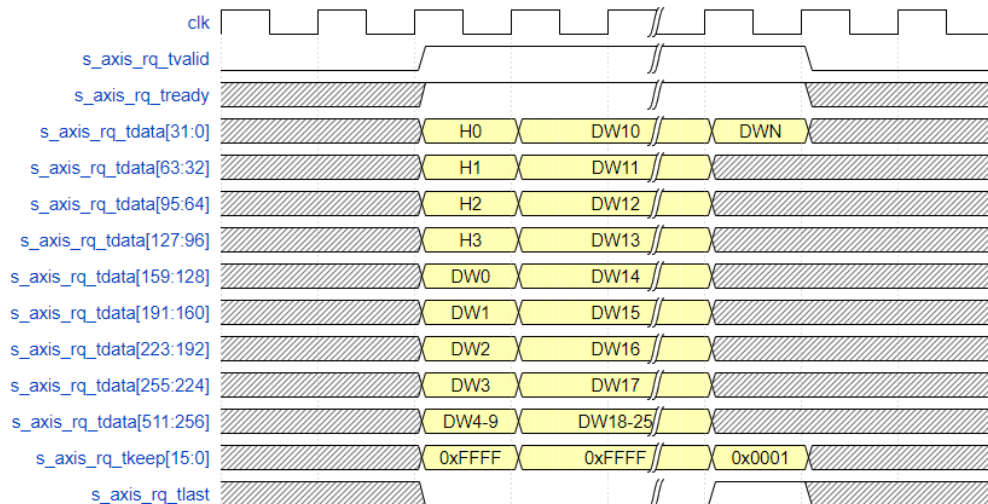


图 3-27 RQ 接口时序图 (512bit)

3.3 顶层信号

工程顶层信号如下表所示：

表 3-28 EP 端顶层信号

名称	I/O	位置	描述
sys_clk	输入	AB15	FPGA 输入时钟，50MHz
sys_rst	输入	K10	FPGA 输入复位，demo 板上 PCIe 金手指 perst 信号
debug_led[0]	输出	W13	FPGA 输入复位，demo 板上 PCIe 金手指 perst 信号
debug_led[1]	输出	W12	PCIe IP user_link_up 输出
debug_led[2]	输出	W15	PLL lock 信号
debug_led[3]	输出	Y16	native2axis_subsys 模块 cfg_ltssm_state[4]
debug_led[4]	输出	Y13	native2axis_subsys 模块 cfg_ltssm_state[3]
debug_led[5]	输出	AA13	native2axis_subsys 模块 cfg_ltssm_state[2]
debug_led[6]	输出	AD14	native2axis_subsys 模块 cfg_ltssm_state[1]
debug_led[7]	输出	AD13	native2axis_subsys 模块 cfg_ltssm_state[0]



4 仿真

参考设计提供 modelsim 仿真工程，并包含了多个测试 case，如表 4-1 所示。在 tb/sim 目录下，运行仿真脚本进行仿真。

表 4-1 测试 case

名称	描述
case1_0	RC 发起 256 次 MEM 写通过 CQ 接口将数据写入 APP 的 bram 中，再发起 256 次 MEM 读通过 CC 接口将数据读回校验，读写数据长度由 1~256byte 递增，地址跟随长度变。
case1_1	RC 发起 256 次 MEM 写通过 CQ 接口将数据写入 APP 的 bram 中，再发起 256 次 MEM 读通过 CC 接口将数据读回校验，读写数据长度为 1byte，地址跟随长度变。
case1_2	RC 发起 64 次 MEM 写通过 CQ 接口将数据写入 APP 的 bram 中，再发起 64 次 MEM 读通过 CC 接口将数据读回校验，读写数据长度为随机长度，地址跟随长度变。
case11_0	RC 配置 EP 中 DMA 寄存器，EP 通过 RQ RC 接口发起 256 次 h2c 方向 DMA，将数据存入 bram，再通过 RQ RC 接口发起 256 次 c2h 方向 DMA，将数据发回 RC 进行数据校验，读写数据长度由 1~256byte 递增，地址跟随长度变。
case11_1	RC 配置 EP 中 DMA 寄存器，EP 通过 RQ RC 接口发起 256 次 h2c 方向 DMA，将数据存入 bram，再通过 RQ RC 接口发起 256 次 c2h 方向 DMA，将数据发回 RC 进行数据校验，读写数据长度为 1byte，地址跟随长度变。
case11_2	RC 配置 EP 中 DMA 寄存器，EP 通过 RQ RC 接口发起 64 次 h2c 方向 DMA，将数据存入 bram，再通过 RQ RC 接口发起 64 次 c2h 方向 DMA，将数据发回 RC 进行数据校验，读写数据长度为随机长度，地址跟随长度变。

4.1 运行仿真

启动 modelsim，在 tb/sim 目录下，输入命令 `do sim.do [CASE_NUM]`，例如 `do sim.do CASE1_0`，仿真完成后 log 如下，用户可以自行添加波形查看。仿真完成后可以输入命令 `./compare_output_result [CASE_NUM]` 进行数据比对，例如 `./compare_output_result CASE1_0`，打印如下 log 代表数据比对成功。要注意，用户在进行仿真之前，需要修改工程相关的路径设定，在 tb/sim/中，需要在 `ph1a400_lib.f`，`ph2a500_lib.f` 和 `native2axis_src.f` 中重新设定源文件或者宏定义文件的路径。



```
# CASE1 read addr = 000000000000297, length = 000000df byte(s)!
# CASE1 read addr = 000000000000376, length = 000000e0 byte(s)!
# CASE1 read addr = 000000000000456, length = 000000e1 byte(s)!
# CASE1 read addr = 000000000000537, length = 000000e2 byte(s)!
# CASE1 read addr = 000000000000619, length = 000000e3 byte(s)!
# CASE1 read addr = 0000000000006fc, length = 000000e4 byte(s)!
# CASE1 read addr = 0000000000007e0, length = 000000e5 byte(s)!
# CASE1 read addr = 0000000000008c5, length = 000000e6 byte(s)!
# CASE1 read addr = 0000000000009ab, length = 000000e7 byte(s)!
# CASE1 read addr = 000000000000aa2, length = 000000e8 byte(s)!
# CASE1 read addr = 000000000000b7a, length = 000000e9 byte(s)!
# CASE1 read addr = 000000000000c63, length = 000000ea byte(s)!
# CASE1 read addr = 000000000000d4e, length = 000000eb byte(s)!
# CASE1 read addr = 000000000000e38, length = 000000ec byte(s)!
# CASE1 read addr = 000000000000f24, length = 000000ed byte(s)!
# CASE1 read addr = 00000000000090ed, length = 000000ee byte(s)!
# CASE1 read addr = 00000000000091db, length = 000000ef byte(s)!
# CASE1 read addr = 00000000000092ca, length = 000000f0 byte(s)!
# CASE1 read addr = 00000000000093ba, length = 000000f1 byte(s)!
# CASE1 read addr = 00000000000094ab, length = 000000f2 byte(s)!
# CASE1 read addr = 000000000000959d, length = 000000f3 byte(s)!
# CASE1 read addr = 0000000000009698, length = 000000f4 byte(s)!
# CASE1 read addr = 0000000000009784, length = 000000f5 byte(s)!
# CASE1 read addr = 0000000000009879, length = 000000f6 byte(s)!
# CASE1 read addr = 000000000000996f, length = 000000f7 byte(s)!
# CASE1 read addr = 0000000000009a6e, length = 000000f8 byte(s)!
# CASE1 read addr = 0000000000009b5e, length = 000000f9 byte(s)!
# CASE1 read addr = 0000000000009c57, length = 000000fa byte(s)!
# CASE1 read addr = 0000000000009d51, length = 000000fb byte(s)!
# CASE1 read addr = 0000000000009e4c, length = 000000fc byte(s)!
# CASE1 read addr = 0000000000009f48, length = 000000fd byte(s)!
# CASE1 read addr = 000000000000a0fd, length = 000000fe byte(s)!
# CASE1 read addr = 000000000000a17b, length = 000000ff byte(s)!
# CASE1 read addr = 000000000000a2fa, length = 00000100 byte(s)!
# CaseN write back done!
# Start to save data, and compare results!
# Data dump end!
# ** Note: $finish : ../bfm/ip_datarw.sv(268)
# Time: 1435796 ns Iteration: 0 Instance: /tb_native2axis_fsm/u_ip_datarw
```

图 4-2 仿真结果

```
# -----
#
# Func: CASE1_0 result compare
#
# -----
#
# CASE1_0 TEST PASS!
# -----
```

图 4-3 数据比对结果

4.2 仿真结构

参考设计中的仿真用例演示了：

- 配置读/写
- Memory 任意长度读/写

Testbench 的顶层模块图如下：

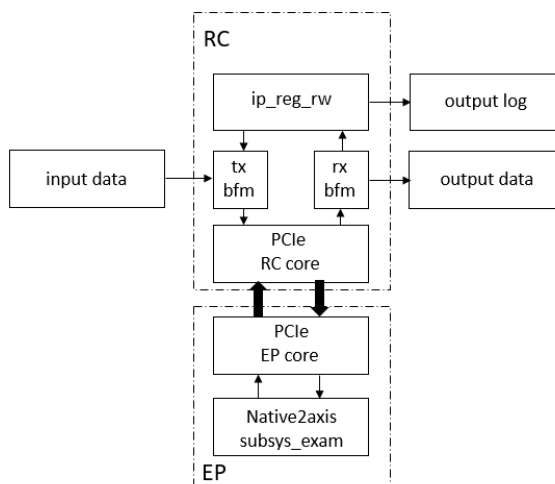


图 4-4 仿真结构框图

其中，ip_reg_rw 模块控制仿真流程并输出仿真 log，tx_bfm 模块从输入数据文件中获得数据，向 EP 端发起 MEM 读写，CFG0 读写和回复完成包，rx_bfm 接收来自 EP 的 MEM 读写请求和完成包，并将完成包数据写入输出数据文件中，最终脚本比对输入数据文件和输出数据文件，完成仿真。



5 编译与调试

5.1 配置说明

实现 RC 所在的 DEMO 板将为 RC 和 EP 提供参考时钟。 请注意检查。

- RC 所在 DEMO 板 R208/R209 必须焊上，即由本地 PLL 产生参考时钟，RC 代码中已经包含了该部分，DEMO 板 R208/R209 位置如下图：



图 5-1 RC 端 R208/R209 位置

- EP 所在 DEMO 板无需修改；

上述两个电阻在电路板上的位置在 Demo 板正面 PCIe 金手指附近。

5.2 RC 工程说明

RC 工程顶层信号如下表所示：

表 5-2 RC 端顶层信号

名称	I/O	位置	描述
app_auxclk	输入	U27	FPGA 输入时钟，50MHz
app_button_rst_n	输入	L13	FPGA 输入复位，Demo 上 SW0
pcie_perest_o	输出	T25	PCIe 金手指 perest 引脚
uart_rx	输入	R24	Uart 接收管脚
uart_tx	输出	U30	Uart 发送管脚
G_led1	输出	J11	PCIe IP rd1h_link_up 信号输出



G_led2	输出	J13	置 0
G_led3	输出	K11	置 0
G_led4	输出	C11	置 0
R_led1	输出	J14	rx_checksum.v 模块 rx_checksum_err 信号输出
R_led2	输出	H11	置 0
R_led3	输出	J12	置 0
R_led4	输出	J17	置 0
intrb_n	输入	T28	RC 端 100M 参考时钟生成，配置外部 si5345 晶振。
Rstb_n	输出	T26	
Sda_io	双向	R30	
OEB	输出	R26	
Sck_o	输出	R28	
SI5345_FINC	输出	P28	
SI5345_FDEC	输出	T30	
SI5345_IN_SEL1	输出	T27	
SI5345_IN_SELO	输出	U28	

用户在 PC 上可以通过串口助手软件，根据串口协议格式发送命令，RC 工程解析命令后再将 MEM 读写 TLP 或者 CFG0 读写 TLP 通过 RC core 发送到对端 EP。EP 返回的完成包由 RC 工程解析后按照串口协议格式发回到 PC 端。自定义串口协议格式如图 5-3 所示。

字节位置	byte0	byte1	byte2	byte3	byte4	byte5~byteN	byte6	byte7
协议解析	帧头	type	Tlp_type	byte5~byteN字节长度 (msb)	byte5~byteN字节长度 (lsb)	地址+长度+数据	校验	帧尾
	0xBB	0x00: RC2EP 0x01: EP2RC	0x00:MEMR 0x01:MEMW 0x02:CFG0R 0x03:CFG0W				byte1~byteN累加和的最低字节	0x7E
MEMW举例	0xBB	0x00	0x01	0x00	0x0A	0x80 0x00 0x00 0x00 0x00 0x04 0x12 0x34 0x56 0x78	0x99	0x7E

图 5-3 串口协议格式

5.3 RC 连接 EP 测试过程

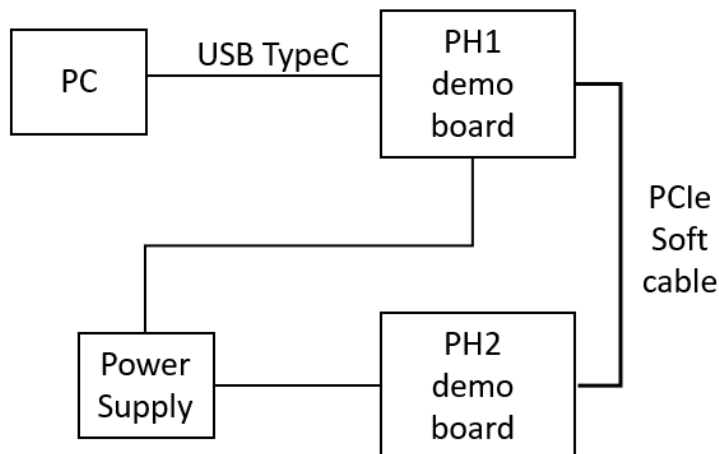


图 5-4 RC 连接 EP 拓扑图

RC-EP 对连拓扑如上图所示。图中的 RC 和 EP 分别是 1 块 PH1A AP103 DEMO Board 和 1 块 PH2A AP201 demo，2 板互连用 PCIe 软排线。JTAG cable 用于板上 FPGA 的下载。USB Type-C 是用于连接 PC 和 RC，因为 DEMO Board 上有 USB 转 UART 的芯片，USB Type-C 连接会被 PC 识别成 COM 口。PC 将通过 COM 口，发起任意长度寄存器读写操作，RC 中会将寄存器的操作转化为 PCIe 的 TLP 事务。

真实连接如以下图片：



图 5-5 RC 连接 EP

其中，PH1A demo 板被设置为 RC，PH2A demo 板被设置为 EP，两个板卡之间 PCIe 接口，采用 PCIe 延长线的方式，连接两个金手指。

在使用软排线进行对接时，注意 RC 和 EP 板卡 PCIE 的正确对接。需要将 RC 的 PCIE TX 信号对接到 EP 的 PCIE RX 信号，将 RC 的 PCIE RX 信号对接到 EP 的 PCIE TX 信号。

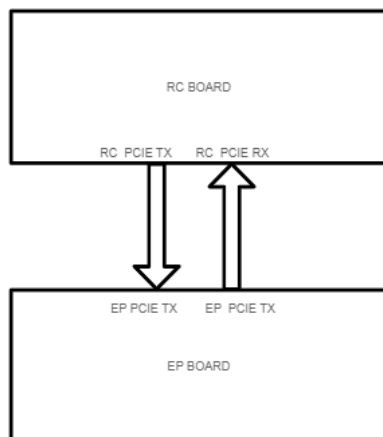


图 5-6 RC 连接 EP 拓扑图

参考设计使用两种不同的 PCIE 延长线。拓扑图如下：

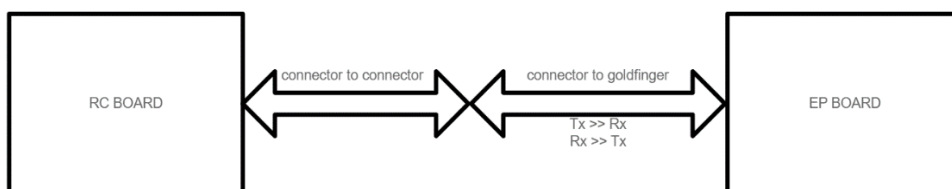


图 5-7 RC 连接 EP 拓扑图 2

先用母对母的延长线连接 RC 板卡和另一延长线的金手指部分，然后在将公转母的母端，连接到 EP。母对母的延长线只有延长信号的作用，无法将 PCIE RX 和 TX 信号对接，而公转母延长线可以将 RX 和 TX 对接。

5.3.1 TLP 测试过程

板卡上电之后，下载 PH1A gen3x4 rc bit 和 PH2A gen3x4 ep bit，用户在测试过程中，可以使用串口调试助手根据串口协议发送数据，也可以直接使用提供的 linux 驱动和脚本进行测试，步骤如下：

1. cd 到 uart2pcie 目录，输入命令 make
2. 生成可执行命令 uart2pcie_test，命令选项如下：

```
-p [usb_device] USB 设备
-d [type] 串口协议中 type 字段
-D [data] 数据
-c [tlp_type] 串口协议中 tlp_type 字段
-a [addr] 地址
```



- l [len] 数据长度
- m [gen_data_type] 数据产生方式 0: random 1:increment 2:decrease
- t [count] 测试循环次数, 加入-t 参数表示进行以下三种测试之一
 - T case1_1 测试选项
 - C case1_2 测试选项
 - 不加-C -T case1_0 测试选项

3. 执行命令 ./cfg.sh, 对 EP 做初始化配置, 配置为使能 master enable, MPS 512byte, BAR0 地址 0x80000000

4. 执行命令 sudo ./uart2pcie_test -p /dev/ttyUSB1 -d 0 -c 0 -a 0x80002000 -m 0 -t 10, 循环跑 10 次 case1_0 测试

5. 执行命令 sudo ./uart2pcie_test -p /dev/ttyUSB1 -d 0 -c 0 -a 0x80002000 -m 0 -t 10 -T, 循环跑 10 次 case1_1 测试

6. 执行命令 sudo ./uart2pcie_test -p /dev/ttyUSB1 -d 0 -c 0 -a 0x80002000 -m 0 -t 10 -C, 循环跑 10 次 case1_2 测试

7 看到如下 log 表明测试成功

```
5a ae 2e d5 e3 41 1f 93 ee c9 73 26 44 3c 61 e8
98 17 9b 80 95 5e 44 e5 c0 c7 3a 87 5b 4c fb b5
fa 29 8a de 6a aa 71 58 73 e4 7e b8 21 e0 a0 b9
f7 3b 3a 8d 99 7e 72 59 45 ad e0 a0 f9 db 56 f4
04 e0 d2 6f 8a 43 c7 fe 27 46 b6 48 26 56 02 1d
91 3c aa 2a ba 1d 84 00 ca 64 a0 c3 40 f6 b7 44
d7 89 b3 61 cc 7b 5f f4 c1 15 3c e7 6b 3e 04 fc
7a af 27 35 cc ab 35 96 0f d5 59 4f cc 11 94 a3
9a 47 04 67 c2 64 5b 83 79 97 6a e5 d6 6f e1 50
1e 08 85 ea b3 ba 80 c3 90 d9 12 5c ea a6 ff 85
ee 03 ec b0
raw buf:
bb 01 00 00 ba 00 00 00 60 00 b4 a3 5c b6 b3 e8
7e c3 c4 50 61 82 55 c7 94 12 74 5a ae 2e d5 e3
41 1f 93 ee c9 73 26 44 3c 61 e8 98 17 9b 80 95
5e 44 e5 c0 c7 3a 87 5b 4c fb b5 fa 29 8a de 6a
aa 71 58 73 e4 7e b8 21 e0 a0 b9 f7 3b 3a 8d 99
7e 72 59 45 ad e0 a0 f9 db 56 f4 04 e0 d2 6f 8a
43 c7 fe 27 46 b6 48 26 56 02 1d 91 3c aa 2a ba
1d 84 00 ca 64 a0 c3 40 f6 b7 44 d7 89 b3 61 cc
7b 5f f4 c1 15 3c e7 6b 3e 04 fc 7a af 27 35 cc
ab 35 96 0f d5 59 4f cc 11 94 a3 9a 47 04 67 c2
64 5b 83 79 97 6a e5 d6 6f e1 50 1e 08 85 ea b3
ba 80 c3 90 d9 12 5c ea a6 ff 85 ee 03 ec b0 3a
7e

recv len: 180, len 180
cnt: 9, mem_rw_stress_test successful...
```

图 5-8 RC 对接 EP 测试 log

5.4 PC 连接 EP 测试过程

PC 连接 EP, 配合 linux 驱动和测试脚本, 可用于对 RQ, RC 接口的测试, al_ep_app 模块中含有 block DMA 控制器, 可主动通过 RQ, RC 接口发起 DMA 传输。



5.4.1 TLP 测试过程

1. 将烧入 bit 的板卡插入 PC 的 PCIe 卡槽中，等待进入系统之后，使用 `lspci` 查看 PCIe 设备信息，如图看到 Device 1edb:abcd 即为 PH2A 的 PCIe 设备。

```
[root@localhost anlogic_pcie_app]# lspci
00:00.0 Host bridge: Intel Corporation Xeon E3-1200 v5/E3-1500 v5/6th Gen Core Processor Host Bridge/DRAM Registers (rev 07)
00:01.0 PCI bridge: Intel Corporation Xeon E3-1200 v5/E3-1500 v5/6th Gen Core Processor PCIe Controller (x16) (rev 07)
00:02.0 VGA compatible controller: Intel Corporation HD Graphics 530 (rev 06)
00:14.0 USB controller: Intel Corporation 100 Series/C230 Series Chipset Family USB 3.0 xHCI Controller (rev 31)
00:14.2 Signal processing controller: Intel Corporation 100 Series/C230 Series Chipset Family Thermal Subsystem (rev 31)
00:16.0 Communication controller: Intel Corporation 100 Series/C230 Series Chipset Family MEI Controller #1 (rev 31)
00:17.0 SATA controller: Intel Corporation Q170/Q150/B150/H170/H110/Z170/CM236 Chipset SATA Controller [AHCI Mode] (rev 31)
00:1c.0 PCI bridge: Intel Corporation 100 Series/C230 Series Chipset Family PCI Express Root Port #5 (rev f1)
00:1c.6 PCI bridge: Intel Corporation 100 Series/C230 Series Chipset Family PCI Express Root Port #7 (rev f1)
00:1d.0 PCI bridge: Intel Corporation 100 Series/C230 Series Chipset Family PCI Express Root Port #9 (rev f1)
00:1f.0 ISA bridge: Intel Corporation H110 Chipset LPC/eSPI Controller (rev 31)
00:1f.2 Memory controller: Intel Corporation 100 Series/C230 Series Chipset Family Power Management Controller (rev 31)
00:1f.3 Audio device: Intel Corporation 100 Series/C230 Series Chipset Family HD Audio Controller (rev 31)
00:1f.4 SMBus: Intel Corporation 100 Series/C230 Series Chipset Family SMBus (rev 31)
01:00.0 Non-VGA unclassified device: Device 1edb:abcd
02:00.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL8111/8168/8411 PCI Express Gigabit Ethernet Controller (rev 15)
03:00.0 Network controller: Qualcomm Atheros QCA9565 / AR9565 Wireless Network Adapter (rev 01)
04:00.0 PCI bridge: Integrated Technology Express, Inc. IT8893E PCIe to PCI Bridge (rev 41)
```

图 5-9 lspci 信息

2. 获得 root 权限，使用 `lspci -vv 1edb:abcd` 查看 PCIe 设备具体信息，从图中可知，设备的 BAR 地址被分配为了 0x6003000000

```
0000:01:00.0 Non-VGA unclassified device: Device 1edb:abcd
Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR- FastB2B- DisINTx-
Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
Region 0: Memory at 6003000000 (64-bit, prefetchable) [disabled] [size=1M]
Capabilities: [80] Express (v2) Endpoint, MSI 00
DevCap: MaxPayload 512 bytes, PhantFunc 0, Latency L0s <64ns, L1 <1us
ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset- SlotPowerLimit 75.000W
DevCtl: CorrErr- NonFatalErr- FatalErr- UnsupReq-
RxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop+
MaxPayload 256 bytes, MaxReadReq 512 bytes
DevSta: CorrErr+ NonFatalErr- FatalErr- UnsupReq+ AuxPwr- TransPend-
LnkCap: Port #0, Speed 16GT/s, Width x8, ASPM not supported
ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp+
LnkCtl: ASPM Disabled; RCB 64 bytes, Disabled- CommClk+
ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
LnkSta: Speed 16GT/s (ok), Width x8 (ok)
TrErr- Train- SlotClk+ DLActive- BWMgmt- ABWMgmt-
DevCap2: Completion Timeout: Not Supported, TimeoutDis+ NROPrPrP- LTR+
10BitTagComp+ 10BitTagReq+ OBFF Via message/WAKE#, ExtFmt+ EETLPPrefix+, MaxEETLPPrefix-
EmergencyPowerReduction Not Supported, EmergencyPowerReductionInit-
FRS- TPHComp+ ExtTPHComp+
AtomicOpsCap: 32bit+ 64bit+ 128bitCAS+
DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis- LTR+ OBFF Disabled,
AtomicOpsCtl: ReqEn-
LnkCap2: Supported Link Speeds: 2.5-16GT/s, Crosslink+ Retimer+ 2Retimers+ DRS-
LnkCtl2: Target Link Speed: 16GT/s, EnterCompliance- SpeedDis-
Transmit Margin: Normal Operating Range, EnterModifiedCompliance- ComplianceSOS-
Compliance De-emphasis: -6dB
LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete+ EqualizationPhase1+
EqualizationPhase2+ EqualizationPhase3+ LinkEqualizationRequest-
Retimer- 2Retimers- CrosslinkRes: Upstream Port
Capabilities: [e0] MSI: Enable- Count=1/1 Maskable- 64bit+
Address: 0000000000000000 Data: 0000
Capabilities: [f8] Power Management version 3
Flags: PMECLK- DSI+ D1- D2- AuxCurrent=0mA PME(D0-,D1-,D2-,D3hot-,D3cold-)
```

图 5-10 lspci 详细信息

3. 进入到 `anlogic_pcie_drv` 目录，执行 `make` 进行编译，然后执行 `insmod anlogic_pci.ko`
4. 进入到 `anlogic_pcie_app` 目录，使用 `gcc dmaMemRW.c -o dmaMemRw` 编译用户函数
5. 使用 `./dmaMemRw 0 0 0 0` 命令，查看驱动的 h2c buffer 和 c2h buffer 地址，如图所示 c2hbuffer 地址为 0x79100000，h2c buffer 地址为 0x79200000



```
[root@localhost anlogic_pcie_app]# ./dmaMemRw 0 0 0 0
Dma data memory base address:
c2h buf address: 0x79100000
h2c buf address: 0x79200000
device /dev/ANLOGIC-PCI0_xbmd_rxmem opened.
c2h memory dump(sz: 0 bytes):
```

图 5-11 驱动 buffer 地址

- 使用命令 `./dma.sh [test_case][h2c buffer address][c2h buffer address][bar0 address]` 开始测试 case，其中 test_case 选项 0 代表 case11_0，1 代表 case11_1，2 代表 case11_2。
- 看到如图所示打印 `log test pass` 证明测试成功

```
00007f20 c0 ea b5 23 48 37 a1 29 14 74 0f 66 c3 9c a4 ab
00007f30 d0 7c db 04 15 1a 8f ef 08 04 85 a0 4c 52 59 d9
00007f40 b1 1c 88 4a 59 ac b0 0f 84 7d b0 7e 42 b8 12 ff
00007f50 89 b1 e3 a7 49 2d 76 a7 74 65 5f a7 33 de ee 6f
00007f60 a2 8f e5 56 5e b0 9d 08 94 34 9c 3c 86 74 77 75
00007f70 f1 de 5f ba d9 1d d7 9c 0d 55 02 b0 51 29 80 d4
00007f80 b7 40 94 14 46 a9 de 5d a5 31 57 f2 67 dc 3a da
00007f90 16 71 4e 39 f5 1e 58 c1 e8 d7 a5 cc 45 28 ca a7
00007fa0 3d b2 ef bd cc 0d b9 05 e9 ac 90 c2 a5 56 87 d0
00007fb0 52 9b 24 49 2d 73 99 f5 f9 fd 00 1c d1 9e a3 17
00007fc0 db 64 11 35 e1 e5 2e c1 49 36 cd af 9a c0 ee 6f
00007fd0 d1 dd e9 2c f4 84 36 f8 df 9b 07 42 4c 04 a3 a1
00007fe0 d3 96 a2 ee bd 06 f4 c4 7c 60 eb 60 2e 1c bb 94
00007ff0 2a 89 eb 45 9d e5 43 54 9c fb 0c b4 14 cf 77 a9
00008000 7b c4 44 53 59 f3 30 86 70 e3 4e 80 5d f3 9b 8c
00008010 2c 76 00 7f 91 81 2b 37 05 21 97 a8 80 44 24 2a
00008020 04 61 9e c4 9c fa f9 1d 77 d5 ef a7 8a 3c 55 9e
00008030 3f 5a af 69 91 13 93 2b 11 f5 8d 5d 59 77 94 87
00008040 61 aa 4f 15 ee 42 95 ec 56 b8 b3 13 5c 2a 84 74
00008050 ae 0c 8c a6 67 fc 2d 3c 78 a9 11 65 74 42 51 2c
00008060 25 b2 00 a3 92 8e 52 90 61 ce 85 09 cf 3a bf 09
00008070 e6 b0 ad ea 7a 75 c0 14 4a f7 aa 70 60 e0 15 05
test pass
```

图 5-12 PC 连接 EP 测试 log



6 工程移植注意事项

当用户需要将参考设计移植到用户工程中时,可以将 native2axis_subsys 模块移植到用户工程中,实现的顶层接口中,时钟和复位相关端口定义如表 6-1 所示,中断和配置相关端口定义如表 6-2 所示,AXIS 端口定义参考 3.2.2 节

表 6-1 native2axis_subsys 模块时钟和复位端口定义

端口	I/O	位宽	说明
tl_clk_user	输入	1	PCIe core 的输入时钟,时钟频率应和 tl clock frequency 一致
tl_power_on_rstn	输入	1	模块复位,低有效
hpcclk_g	输入	1	HXT APB 总线配置时钟,必须小于 100MHz
tl_rstn_user	输出	1	用户逻辑复位,低有效

表 6-2 native2axis_subsys 模块中断和配置端口定义

端口	I/O	位宽	说明
cfg_interrupt_int	输入	4	用于设置 INTx 中断,高有效 bit[0]: INTA# bit[1]: INTB# bit[2]: INTC# bit[3]: INTD#
cfg_interrupt_sent	输出	1	中断已发出标志信号,高有效
cfg_interrupt_msi_enable	输出	4	MSI 使能信号,置 1 表示对端使能 MSI 中断
cfg_interrupt_msi_int	输入	32	MSI 中断向量
cfg_interrupt_msi_function_number	输入	8	MSI 中断 function 编号,有效值 0~3
cfg_interrupt_msi_sent	输出	1	MSI 中断已发出标志信号,高有效
cfg_interrupt_msi_mmenable	输出	12	Multiple Message Enable



			000b: 1 vector requested 001b: 2 vectors requested 010b: 4 vectors requested 011b: 8 vectors requested 100b: 16 vectors requested 101b: 32 vectors requested 110b: 保留 111b: 保留
cfg_interrupt_msi_pending_status	输入	32	MSI Pending
cfg_interrupt_msi_pending_status _function_num	输入	2	MSI Pending function 编号
cfg_interrupt_msi_pending_status _data_enable	输入	1	MSI Pending 有效信号
cfg_interrupt_msi_mask_update	输出	1	MSI mask 更新标志信号, 高有效
cfg_interrupt_msi_select	输入	2	MSI function 选择信号
cfg_interrupt_msi_data	输出	32	MSI mask 信号
user_link_up	输出	1	PCIe core link up 信号, 高有效
cfg_link_training_enable	输入	1	使能 ltssm
cfg_ltssm_state	输出	6	ltssm 状态机, ltssm 状态解码如下: 00h: detect.quiet 01h: detect.active 02h: polling.active 03h: polling.compliance 04h: polling.configuration 05h: config.linkwidthstart



			06h: config.linkwidthaccept 07h: config.lanenumwait 08h: config.lanenumaccept 09h: config.complete 0Ah: config.idle 0Bh: recovery.receiverlock 0Ch: recovery.equalization 0Dh: recovery.speed 0Eh: recovery.receiverconfig 0Fh: recovery.idle 10h: L0 11h: L0s 12h: L1.entry 13h: L1.idle 14h: L2.idle/L2.transmit wake 15h: 保留 16h: disable 17h: loopback.entry 18h: loopback.active 19h: loopback.exit 1Ah: hotreset
cfg_rcb_status	输出	4	读完成边界 0000b:64 字节 1111b:128 字节
cfg_bus_number	输出	8	PCIe 总线编号
cfg_negotiated_width	输出	3	当前协商链路位宽:



			000b:x1 001b:x2 010b:x4 011b:x8
cfg_current_speed	输出	2	当前协商链路速度: 00b:gen1 01b:gen2 10b:gen3 11b:gen4
cfg_max_payload	输出	2	当前链路最大有效负载值 00b:128 字节 01b:256 字节 10b:512 字节 11b:1024 字节
cfg_max_read_req	输出	3	当前协商最大读请求值 000b: 128 字节 (默认值) 001b: 256 字节 010b: 512 字节 011b: 1024 字节 100b: 2048 字节 101b: 4096 字节 110b: 保留 111b: 保留

参考设计默认提供 PCIe gen4x8 工程，当用户需要在参考设计中修改 PCIe 链路速率配置时，例如从 gen4x8 修改到 gen1x1 时，首先需要打开 PCIe ip gen，将链路速率修改为 gen1x1，并选择需要输出的 tl clock 频率 62.5MHz，并修改 tl_clk_user_freq 端口数值为 9'd63，然后打开 PLL ip gen，将



输出时钟频率调整为在 PCIe ip gen 中选择的 tl clock 频率 62.5MHz，最后修改 SDC 文件中 sys_clk_g 时序约束为 62.5MHz。

参考设计默认 AXIS 接口数据位宽 512bit，当用户修改 AXIS 接口数据位宽时，需要将 para_def.vh 文件中的 `ODATA\_WIDTH，`CQ_TUSER_WIDTH，`RC\_TUSER\_WIDTH，`CC_TUSER_WIDTH，`RQ_TUSER_WIDTH 宏定义修改到对应的数值。



7 参考文献

1. 《UG916_安路科技 PH1A400SFG900 DEMO 板使用说明》
2. 《UG913_安路科技 PH1A400 系列 FPGA PCIE 用户手册》
3. 《PH1_DEMO_Board_si5344/si5342 User Guide》
4. 《UG1112_安路科技 PH2A500 系列 FPGA PCIE 用户手册》



版本信息

日期	版本	修订记录
2024. 5. 23	1. 0	首次发布正式版

版权所有©2024 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其他方式授予任何知识产权许可；本文档仅为向用户提供使用器件的参考，协助用户正确地使用安路科技产品之用，其著作权归安路科技所有；本文档所展示的任何产品信息均不构成安路科技对所涉产品或服务作出任何明示或默示的声明或保证。

安路科技将不定期地对本文档进行更新、修订。用户如需获取最新版本的文档，可通过安路科技的官方网站（网址为：<https://www.anlogic.com>）自行查询下载，也可联系安路科技的销售人员咨询获取。